

Competitive Programming

Section 45: Flows and Matchings

Competitive Programming Course

2026-07-06

Table of contents

1	Flow as controlled movement through a network	3
1.1	Flow definitions	3
1.2	Dinic's algorithm	3
1.3	Bipartite matching as flow	4
1.4	Minimum cut after max flow	4
1.5	Implementation checklist	4
1.6	Tiny max-flow trace	5
1.7	Residual edge meaning	5
1.8	Flow debugging tests	5
2	Exercises	6

1 Flow as controlled movement through a network

A flow network has capacities on directed edges. Dinic's algorithm repeats: build a BFS level graph, then send blocking flow by DFS along increasing levels. Bipartite matching is a special case of flow, and a minimum cut is the set of vertices still reachable from the source in the residual graph after max flow.

1.1 Flow definitions

Each directed edge (u, v) has capacity $c(u, v)$. A feasible flow assigns a number $f(u, v)$ to each edge such that:

- $0 \leq f(u, v) \leq c(u, v)$;
- every vertex except source s and sink t has equal incoming and outgoing flow.

The value of the flow is the total amount leaving s .

Residual capacity tells us how we can still change the flow:

```
forward residual:  $c(u, v) - f(u, v)$ 
backward residual:  $f(u, v)$ 
```

The backward edge is essential. It represents undoing or rerouting previous flow.

1.2 Dinic's algorithm

Dinic works in phases.

1. BFS from s using only positive residual edges. This gives $level[v]$.
2. DFS sends flow only along edges where $level[to] == level[v] + 1$.
3. The DFS continues until no more flow can be sent in this level graph.
4. Repeat BFS.

The level graph prevents DFS from wandering in cycles and gives a clear progress measure. The pointer array $it[v]$ remembers which outgoing edges have already been exhausted during the current BFS phase.

Diagram:

```

s --level 0
| \
v v
level 1 nodes
|   \
v     v
level 2 nodes ... t

```

DFS only moves left-to-right through increasing levels.

1.3 Bipartite matching as flow

For a bipartite graph with left side L and right side R :

```

source -> every left vertex      capacity 1
left u -> right v if edge exists capacity 1
every right vertex -> sink       capacity 1

```

Every unit of flow corresponds to one matched pair. Capacity 1 ensures each vertex is used at most once.

For the local matching exercise we use the classic augmenting-path DFS because it is shorter and exposes the same idea: try to give a left vertex a right partner, recursively moving an already matched left vertex if possible.

1.4 Minimum cut after max flow

After max flow, run BFS/DFS from s in the residual graph. Let S be the set of reachable vertices. There is no residual edge from S to outside S ; otherwise the outside vertex would be reachable. The saturated original edges crossing from S to $V-S$ form a minimum cut, and its capacity equals the max flow value.

This is the max-flow min-cut theorem in the form most useful in contests.

1.5 Implementation checklist

- Store reverse-edge indices, not just reverse endpoints.
- When sending x through an edge, subtract x from the forward residual and add x to the reverse residual.
- Use `long long` for capacities and total flow in C++.
- Rebuild `level` every phase.
- Reset it after each BFS.

1.6 Tiny max-flow trace

Network:

```
s -> a capacity 3
s -> b capacity 2
a -> t capacity 2
b -> t capacity 4
a -> b capacity 1
```

One maximum flow has value 4:

```
s->a->t sends 2
s->b->t sends 2
```

The remaining capacity from **a** to **b** does not help because **s**->**a** has only one extra unit left and **b**->**t** has room, so another unit can go:

```
s->a->b->t sends 1
```

Total becomes 5, which is also the total capacity leaving **s** ($3+2$), so it is maximum. This small example shows why residual paths may combine edges that were not part of the first obvious augmenting paths.

1.7 Residual edge meaning

If an original edge has capacity 7 and currently carries flow 5, the residual graph contains:

```
forward capacity 2
backward capacity 5
```

The backward capacity does not mean the original graph had a reverse road. It means the algorithm may cancel up to 5 units of previous flow if that enables a better global routing.

1.8 Flow debugging tests

Use tiny graphs:

```
single edge s->t
two parallel paths
bottleneck in the middle
disconnected sink
edge that requires rerouting through a residual edge
```

For matching, test one left vertex connected to many right vertices and many left vertices competing for one right vertex. These cases reveal whether the augmenting DFS actually reassigns matches.

2 Exercises

- A. Max Flow: Given a directed capacitated graph from node 1 to node n , print the maximum flow.
- B. Bipartite Matching: Given edges between left and right parts, print the maximum matching size.
- C. Min Cut Reachable Side: After max flow from 1 to n , print the flow value and the source-side reachable vertices in the residual graph.